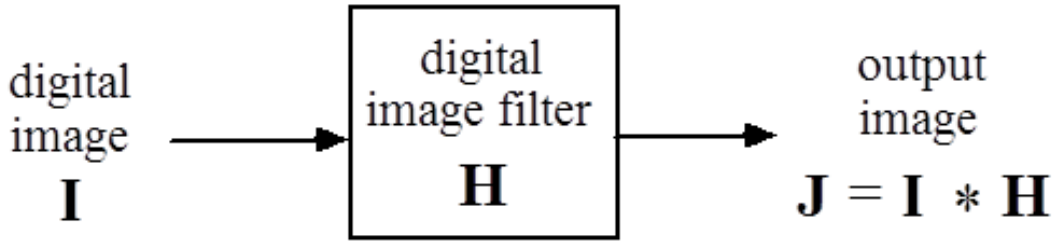
Πανεπιστήμιο Κύπρου
Τμήμα Πληροφορικής

ΕΠΛ445:
Ψηφιακή Επεξεργασία Εικόνας

Εργαστήριο 4

Φίλτρα εικόνων (Συνέλιξη - Convolution)

- Είναι μια διαδικασία που μετασχηματίζει την εικόνα I με τη χρήση γραμμικής συνέλιξης (ευθύς μετασχηματισμός της συνέλιξης):



- **Στόχος:** Επεξεργασία δειγματοληπτημένων και κβαντισμένων εικόνων ώστε να μετασχηματιστούν σε:
  - Εικόνες **καλύτερης ποιότητας** (βάσει συγκεκριμένων κριτηρίων)
  - Εικόνες με **ορισμένα χαρακτηριστικά ενισχυμένα**
  - Εικόνες με **ορισμένα χαρακτηριστικά αποδυναμωμένα ή εξαλειμμένα**

Φιλτράρισμα Εικόνας(Συνέλιξη - Convolution)

Οι λειτουργίες επεξεργασίας εικόνας που υλοποιούνται με φιλτράρισμα περιλαμβάνουν:

- **Εξομάλυνση (Smoothing):** αφαίρεση θορύβου από σφάλματα bit, μετάδοση κ.λπ.
- **Όξυνση (Sharpening):** αύξηση της ευκρίνειας θολών εικόνων
- **Ενίσχυση (Enhancement):** έμφαση σε σημαντικά χαρακτηριστικά, όπως οι ακμές
- Συνδυασμό των παραπάνω

Συνέλιξη 2D Convolution (παράδειγμα)

Εφαρμόζουμε ένα **3×3 average filter** (mean filter) στο pixel που είναι με κόκκινο. Άρα ουσιαστικά υπολογίζουμε **τον μέσο όρο των 9 γειτονικών τιμών**.

Επειδή το pixel είναι στη γωνία, **δεν έχουμε 9 πραγματικές τιμές**. Οπότε χρησιμοποιούμε **zero padding**:
Όπου δεν υπάρχει τιμή εικόνας → βάζουμε 0.

I =

20	10	30
20	10	30
20	10	30

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 20 & 10 \\ 0 & 20 & 10 \end{bmatrix}$$

$K = \frac{1}{9}$

1	1	1
1	1	1
1	1	1

Νέα τιμή pixel = (20*1 + 10*1 + 20*1 + 10*1)/9 = 6.66

Image Filtering (2D Convolution) in OpenCV

```
import numpy as np
import cv2
from matplotlib import pyplot as plt

img = cv2.imread('zebra.jpg',0)

# Create a 5x5 average filter kernel
kernel = np.ones((5,5), np.float32)/25
# Convolve the kernel with the image
dst = cv2.filter2D(img,-1,kernel)

# Create a 9x9 average filter kernel
kernel = np.ones((9,9), np.float32)/81
# Convolve the kernel with the image
dst2 = cv2.filter2D(img,-1,kernel)
```

```
plt.subplot(131),plt.imshow(img, cmap = 'gray')
plt.title('Input Image'), plt.xticks([]), plt.yticks([])
plt.subplot(132),plt.imshow(dst, cmap = 'gray')
plt.title('Average Filter #1'), plt.xticks([]),
   plt.yticks([])
plt.subplot(133),plt.imshow(dst2, cmap = 'gray')
plt.title('Average Filter #2'), plt.xticks([]),
   plt.yticks([])
plt.show()
```

Αλγόριθμος: Κρατώντας τον πυρήνα πάνω από ένα pixel, προσθέστε όλα τα 25 pixels κάτω από αυτό τον πυρήνα, υπολογίστε το μέσο όρο του και να αντικαταστήσει το κεντρικό pixel με τη νέα μέση τιμή.

$$K = \frac{1}{25}\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Depth combinations

Input depth (src.depth())	Output depth (ddepth)
CV_8U	-1/CV_16S/CV_32F/CV_64F
CV_16U/CV_16S	-1/CV_32F/CV_64F
CV_32F	-1/CV_32F/CV_64F
CV_64F	-1/CV_64F

Note

when ddepth=-1, the output image will have the same depth as the source.

Input Image

Average Filter #1

Average Filter #2

Image Smoothing: Averaging

```
import cv2
from matplotlib import pyplot as plt

img = cv2.imread('zebra.jpg',0)

# Image blurring using averaging (low-pass filter)
blur = cv2.blur(img,(5,5))

# Image blurring having greater dimensions
blur2 = cv2.blur(img,(9,9))

plt.subplot(131),plt.imshow(img, cmap = 'gray')
plt.title('Input Image'), plt.xticks([]), plt.yticks([])
plt.subplot(132),plt.imshow(blur, cmap = 'gray')
plt.title('Blurred #1'), plt.xticks([]), plt.yticks([])
plt.subplot(133),plt.imshow(blur2, cmap = 'gray')
plt.title('Blurred #2'), plt.xticks([]), plt.yticks([])
plt.show()
```

Image Smoothing: Gaussian and Median Blurring

```
import numpy as np
import cv2
from matplotlib import pyplot as plt

img = cv2.imread('zebra.jpg',0)

# Apply low-pass filters
# Gaussian blurring
blur = cv2.GaussianBlur(img,(5,5),0)

# Median blurring
median = cv2.medianBlur(img,5)
```

Kernel size

Kernel size, 5x5

Standard deviation in X and Y directions. If value is zero, they are calculated from kernel size. St.Dev. is used to define the amount of blur.

More info click here

```
plt.subplot(131),plt.imshow(img, cmap = 'gray')
plt.title('Input Image'), plt.xticks([]), plt.yticks([])
plt.subplot(132),plt.imshow(blur, cmap = 'gray')
plt.title('Gaussian'), plt.xticks([]), plt.yticks([])
plt.subplot(133),plt.imshow(median, cmap = 'gray')
plt.title('Median'), plt.xticks([]), plt.yticks([])
plt.show()
```

Median Algorithm: Παίρνει όλες τις τιμές μέσα στο kernel, τις ταξινομεί, και επιλέγει τη μεσαία τιμή.

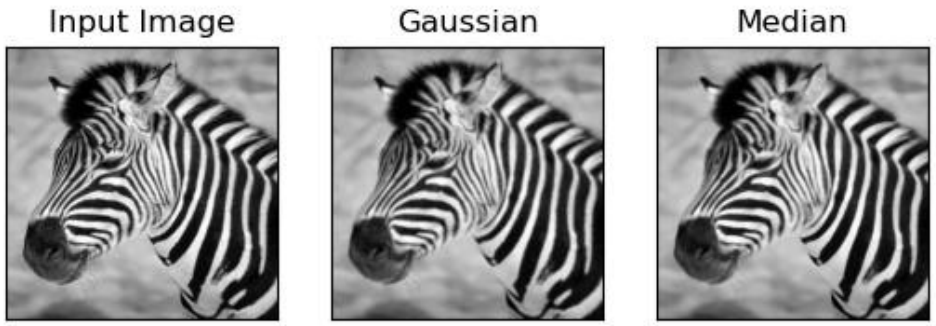


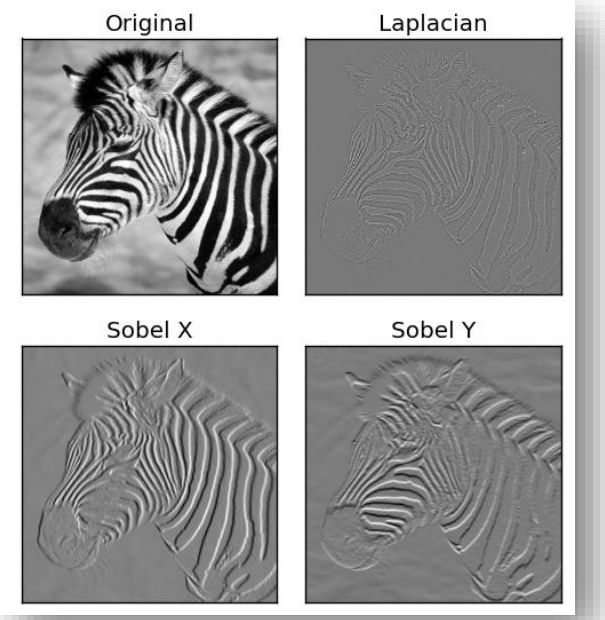
Image Gradients

```
import cv2
from matplotlib import pyplot as plt

img = cv2.imread('zebra.jpg',0)

# Apply image gradients or high-pass filters
laplacian = cv2.Laplacian(img,cv2.CV_64F)
sobelx = cv2.Sobel(img,cv2.CV_64F,1,0,ksize=5)
sobely = cv2.Sobel(img,cv2.CV_64F,0,1,ksize=5)

plt.subplot(2,2,1),plt.imshow(img,cmap = 'gray')
plt.title('Original'), plt.xticks([]), plt.yticks([])
plt.subplot(2,2,2),plt.imshow(laplacian,cmap = 'gray')
plt.title('Laplacian'), plt.xticks([]), plt.yticks([])
plt.subplot(2,2,3),plt.imshow(sobelx,cmap = 'gray')
plt.title('Sobel X'), plt.xticks([]), plt.yticks([])
plt.subplot(2,2,4),plt.imshow(sobely,cmap = 'gray')
plt.title('Sobel Y'), plt.xticks([]), plt.yticks([])
plt.show()
```

Ανίχνευση ασυνέχειας στη φωτεινότητα της εικόνας με τη βοήθεια υπολογισμού της κλίσης (gradient) της εικόνας. Η κλίση της εικόνας είναι το πλάτος του διανύσματος που παρέχει πληροφορία για τα τις μεταβολές της φωτεινότητας.

Το φίλτρο Sobel είναι φίλτρο Gaussian smoothing με μια διαφοροποίηση, έτσι ώστε να είναι πιο ανθεκτικό στον θόρυβο. Μπορούμε να καθορίσουμε την κατεύθυνση των παραγώγων που πρέπει να ληφθούν, κάθετη ή οριζόντια (Yorder και Xorder αντίστοιχα). Μπορούμε επίσης να καθορίσουμε το μέγεθος του πυρήνα.

More info click here

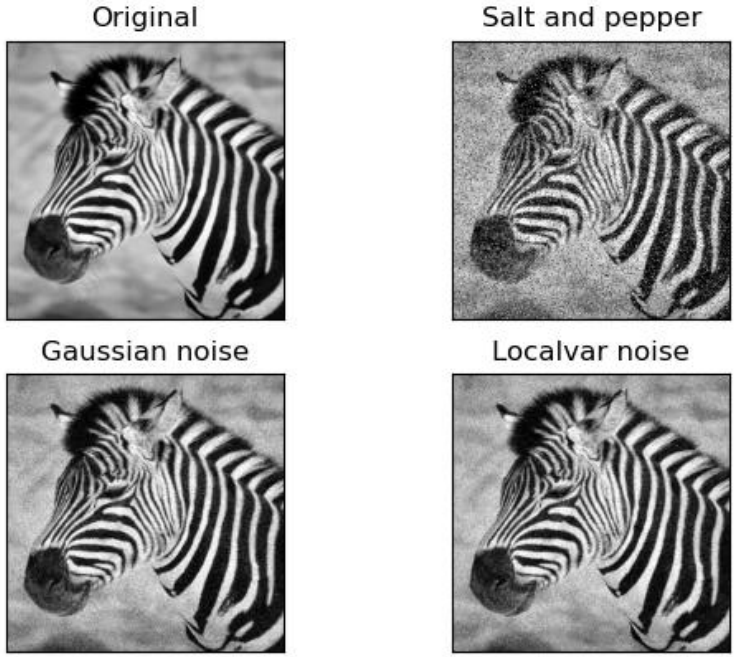


Εισαγωγή Θορύβου

- **gaussian**: Gaussian white noise with constant mean and variance
- **localvar**: Zero-mean Gaussian white noise with an intensity-dependent variance
- **poisson**: Poisson-distributed noise
- **salt**: Replaces random pixels with 1
- **pepper**: Replaces random pixels with 0
- **s&p**: Salt and Pepper - On and off pixels
- **speckle**: Multiplicative noise [out = image + n*image, where n is Gaussian noise with specified mean & variance]

Noise in OpenCV

```
import skimage

import cv2

from matplotlib import pyplot as plt


img = cv2.imread('zebra.jpg',0)


# Apply salt-pepper, gaussian and localvar noise
to the image

s_and_p = skimage.util.random_noise(img, mode="s&p",
amount=0.2)

gauss = skimage.util.random_noise(img, mode="gaussian")

localvar=skimage.util.random_noise(img, mode="localvar")


plt.subplot(2,2,1),plt.imshow(img,cmap = 'gray')

plt.title('Original'), plt.xticks([]), plt.yticks([])

plt.subplot(2,2,2),plt.imshow(s_and_p,cmap = 'gray')
```

More info click here

```
plt.title('Salt and pepper'), plt.xticks([]),
plt.yticks([])

plt.subplot(2,2,3),plt.imshow(gauss,cmap = 'gray')

plt.title('Gaussian noise'), plt.xticks([]), plt.yticks([])

plt.subplot(2,2,4),plt.imshow(localvar,cmap = 'gray')

plt.title('Localvar noise'), plt.xticks([]), plt.yticks([])

plt.show()
```